NTNU
Kunnskap for en bedre verden

Institutt for Teknisk Kybernetikk

TTK4235 - Tilpasset Datasystemer

Heis Lab

Forfattere:
Håkon Karlsen & Aksel Holsvik Eriksen

Gruppe 13

10.03.2026

Table of Contents

List of Figures **i**

1 Introduction **1**

2 Prosjekt Beskrivelse **1**

3 UML Diagram **1**

4 Refleksjonsnotat **2**

4.1 UML og V-modellen 2

4.2 KI 3

4.3 Robusthet, skalarbarhet, og vedlikehold 3

5 Konklusjon **3**

List of Figures

1 Tilstandsdiagram 1

2 Klassediagram 2

1 Introduction

Målet med Heis Labben er å implemetere et styre system for en enkelt heis. Systemet implementerer en tilstandsmaksin samt et køsystem for å opnå ønsket resultat i samsvar med en omfattende Factory Acceptance Test (FAT).

2 Prosjekt Beskrivelse

System-implementasjonen er basert på tilstandsdiagrammet vedlagt i neste underdel. Kø systemet kjører seperat og er implementert som en doble-lenket liste. Når brukeren trukker på en knapp blir en ny node lagt til bakerst i denne listen, dersom en tilsvarende order ikke allerede eksisterer i køen. Rekkefølgen i køen kan anses som en prioritetsrekkefølge, men ordrer kan kan slettes vilkårlig fra listen ut i fra reisen som heisen tar.

Systemet er implementert som en stor polling loop som først sjekker tilstandene til knapper og brukker dette til å oppdatere køen. Videre kjøres tilstandsmaskinen som en switch-case basert på hvilken tilstand heisen er i, og handlinger i tilstanded baserer seg på hvilke knapper som presses og status på ordre-køen.

3 UML Diagram

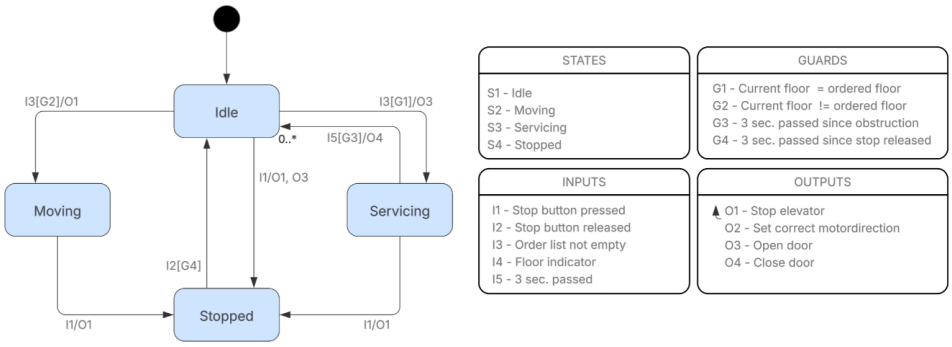

Figure 1: Tilstandsdiagram

Ovenfor er tilstandsdiagramet som beskriver virkemåten til systemets endelige implementasjon. Diagrammet er forholdsvis enkelt, med kun fire tilstander og et begrenset antall overganger mellom dem. Mye av logikken til systemet ligger i triggers, gaurds, og actions som styrer flyten mellom tilstander. Ordre blir håndtert parallelt med tilstandsmaskinen og danner en inngang til tilstandsmaskinen.

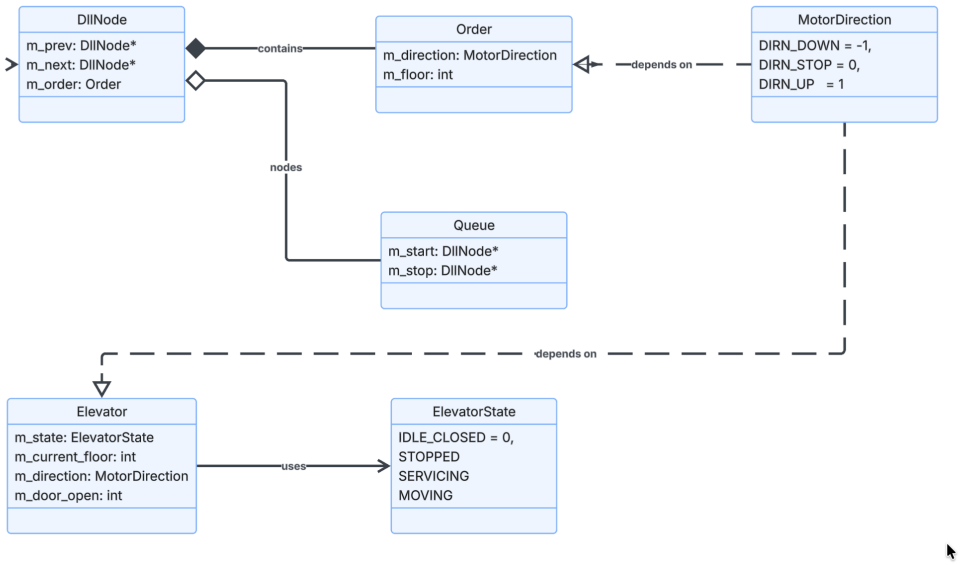

Figure 2: Klassediagram

Structs og enums tatt i bruk i prosjektet er detaljert ovenfor. DllNode, Order, og Queue utgjør datatypene som brukes for å danne et køsystem som baserer seg på en dobbelt-lenket liste, mens Elevator og ElevatorState brukes primært av tilstandsmaskinen.

4 Refleksjonsnotat

4.1 UML og V-modellen

Bruk av UML dannet et godt grunnlag for planleggingen av det endelige produktet. Det gidde oss en grafisk måte å beskrive tilstandsmaksinen vår sammen med programflyten, noe som var nyttig for senere implementasjon. Samtidig opplevde vi at den endelige implementasjonen avviket fra det opprinelige UML diagrammet. Vi oppdaget logiske feil med vårt design og gjord forbedringer underveis. I sluttfasen hadde vi dermed behov for å endre UML diagrammet vårt for å samsvare med vår endelig implementasjon.

En av endringene vi gjord undervies var å omstrukturere tilstandene til systemet. Vi hadde opprinelig en tilstand for MOVING UP og en for MOVING DOWN, men vi oppdaget at logikken for begge tilstandene var tilnærmet identisk. Vi valgte derfor å slå sammen tilstandene til en felles MOVING tilstand. I tilegg endret vi navnet til IDLE OPEN tilstanden til SERVICING ettersom det nye navnet var tydligere i forhold til hensikten med tilstanden.

Vi oppdaget også feil med vår programflyt, men disse feilene var hovedsaklig med selve implementasjonen i kode ovenfor feil med det overordnede systemdesignet.

Vi var ikke særdeles flinke til å planlegge i henhold til V-modellen. Ingen egen-utvilket unit- eller integrationstester ble planlagt før vi begynte å skrive kode. Istedenfor tok vi kun utgangspunkt i de endelige integrasjonstestene som var del av FAT testen. Dette førte til at problem, spesielt med køssystemet, ikke ble oppdaget før de ble testet sammen med resten av systemet. Ofte måtte vi ta flere skritt til bake og teste køsystemet på nytt for å sikre oss at det funket som tiltenkt. Disse feilene skull ha blitt avdekket mye tidligere. I dette tilfellet, spesielt gitt kompleksiteten av vårt løsning for køssystem, burde vi a planlagt mer omfattende unit tests på forhånd i henhold til V modellen.

Å unngå bruken av UML hadde spart oss tid rett og slett fordi vi ikke trenger å bruke tid på å lage

diagramet. Ettersom dette prosjektet ikke var for omfattende kan man klare seg uten dette. Det var nyttig å ha en overordnet ide over hvordan vi skulle gå fram for å implementere programmet, men mye av implementasjonen ble ikke avdekket før man faktisk begynte å programmere. For større prosjekt har man større behov for forhåndsplanlegging, både av programflyt og av hvilke tester som skal brukes for å validere koden. Derimot er det uklart for oss om vi faktisk sparte tid ved å delvis skulke bruken av V-modellen. Vi sparte tid på å drøfte omfattende tester i forveien, men når systemet ikke funket måtte vi uansett skrive slike tester for å verifisere systemet. Det hadde nok da vert mer lønnsomt å bare skrive disse på forhånd.

4.2 KI

Vi har bevist valgt å minimere bruken av KI i prosjektet for å få mest mulig læringsutbytte. All kode har dermed blitt skrevet for hånd. Derimot valgte vi å ta i bruk KI til oppgaver som vi oppfattet som kjedelige, monotone og uten noe særlig læringsgevinst. I dette prosjektet var dette hovedsakelig skriving av doxyblock-ene til headerfilene i koden. Vi brukte Claude til å generere doxyblocks men gikk selv igjennom og manuelt verifiserte at kommenterene som ble skrevet ga mening. Dette gjord at vi kunne spare tid på denne oppgaven men fortsatt opprettholde læringsutbytte for alle andre oppgaver i prosjektet. Vi forsøkte også i størst mulig grad å unngå å stille spørsmål til KI og heller besvare spørsmål ved Google søk eller å henvise oss til dokumentasjon. Med det sagt for man nå til dags KI oppsumeringer av søksresultatet i samme vindu som søksresultatet. Disse oppsumeringene har da hatt en viss innflytelse på vårt arbeid.

4.3 Robusthet, skalarbarhet, og vedlikehold

Vår implementasjon av køssytemet som en doubly linked list førte til problemer med robusthet, ettersom regelmessig bruk av pekere skapte faren for minnelekasjer og lignende problemer. Under testing støttet vi flere ganger på minnelekasjer og use-after-free feil. Disse feilene oppsto kun ved runtime, og ofte kun etter en spesifikk sekvens hadde blitt gjennomført i programflyten. Det gjord feilen vanskelig å oppdage, og det var konstant fare for at en ny minnelekasje kunne oppstå. Det hadde vert mulig å garantere at programmet aldri ville føre til en minnelekasje, men dette hadde krevd mer grundig planlegging. Vi måtte ha kartlagt alle tilfeller der det var fare for minnelekasje og forsikret oss om at minne var frigjort og vi ikke hadde noen hengende pekere. UML hadde vert et nyttig visualiseringsverktøy for denne prossesen.

Testing skulle også ha vert mer gjennomtenkt. Vi burde ha planlagt grundig tester av køsystemet vårt som kunne tvinge fram de minnelekasjene vi var ute etter å avdekke. Dersom køsystemet hadde bestått disse testene kunne vi ha vært sikker på systemet ikke sto i fare for uventet minnelekasjer. Testing ble derimot tatt på sparket, som førte gjentatte ganger til at en uventet minnelekasjer oppsto mens selv etter vi trodde at køsystemet fungerte. Plannlegging i samsvar med V-modellen og UML hadde nok dermed spart oss litt tid her.

På den andre siden av saken opplevde vi det som vannskelig å designe et omfattende UML diagram for systemet vårt som tok høyde for FAT krav og betingelser på en fornuftig måte. Vår opprinelige design inneholdte flere logiske feil som vi ikke klarte å oppdaget før vi forsøkte å implementere systemet i kode. Det ble det naturlig å fortsette alt arbeidet i en IDE istedenfor å oppdatere UML diagram underveis. Det førte potensielt til at vi hadde litt manglende visuell oversikt over systemet vårt.

5 Konklusjon

Vi har implementert et fungerende styresystem for en enkelt heis. Til tross for at utviklingsprossesen ikke var optimal besto styresystemet FAT-testen og kan dermed anses som skikket til bruk.